

Enigma Box

Jujur saya bingung nama tim nya apa
Cryptography

Write-up Penyelesaian

1. Jadi pada challenge ini kita diberikan file yaitu `enigma_box` maka disini saya mencoba menganalisis file nya dan ketika saya cek file nya hasilnya berikut.



```
file enigma_box
enigma_box: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=7d6386b99a081f349b34da57dcd22a7451b28c6a, for GNU/Linux 3.2.0, not stripped
```

Jadi file ini bertipe ELF dan executable

2. Lalu selanjutnya saya disini menggunakan strings untuk mencari text siapa tau ada petunjuk dengan command berikut.

```
strings -n 4 enigma_box | egrep -i 'sha|sha256|aes|iv|cipher|password|key|Pa55|SRIFOTON' -A3
```

ketika dijalankan menghasilkan output berikut.



```
strings -n 4 enigma_box | egrep -i 'sha|sha256|aes|iv|cipher|password|key|Pa55|SRIFOTON' -A3

SHA256_Init
EVP_DecryptFinal_ex
EVP_DecryptInit_ex
EVP_DecryptUpdate
EVP_aes_256_cbc
EVP_CIPHER_CTX_new
EVP_CIPHER_CTX_free
SHA256_Final
SHA256_Update
puts
strlen
__libc_start_main
--
Masukkan password untuk membuka Enigma Box:
%99s
[+] Password Benar!
[*] Menurunkan kunci enkripsi...
[*] Mendekripsi data...
[-] Password Salah. Program berhenti.
;*3$"
#jW"
"3DUfw
--
ciphertext
EVP_DecryptUpdate@OPENSSL_3.0.0
EVP_aes_256_cbc@OPENSSL_3.0.0
SHA256_Init@OPENSSL_3.0.0
EVP_CIPHER_CTX_new@OPENSSL_3.0.0
_fini
__libc_start_main@GLIBC_2.34
EVP_CIPHER_CTX_free@OPENSSL_3.0.0
```

Disini saya menemukan strings yang menarik disini terlihat mungkin menggunakan algoritma apa untuk mendecrypt password nya.

3. Lalu selanjutnya saya disini melihat section `.rodata` / `.data` untuk konstanta (IV / ciphertext)

Disini saya menggunakan `Gunakan readelf` untuk menampilkan isi hex section:

Dengan command berikut :

- `readelf -x .rodata enigma_box | less -S`
- `readelf -x .data enigma_box | less -S`

Ketika saya jalankan command `readelf -x .rodata enigma_box | less -S` mengeluarkan output berikut.

```
Hex dump of section '.rodata':
0x00002000 01000200 00000000 50657361 6e206265 .....Pesan be
0x00002010 72686173 696c2064 6964656b 72697073 rhasil didekrips
0x00002020 693a0a25 730a0000 4d617375 6b6b616e i:.%s...Masukkan
0x00002030 20706173 73776f72 6420756e 74756b20 password untuk
0x00002040 6d656d62 756b6120 456e6967 6d612042 membuka Enigma B
0x00002050 6f783a20 00253939 73000000 00000000 ox: .%99s.....
0x00002060 5b2b5d20 50617373 776f7264 2042656e [+] Password Ben
0x00002070 6172210a 5b2a5d20 4d656e75 72756e6b ar!.[*] Menurunk
0x00002080 616e206b 756e6369 20656e6b 72697073 an kunci enkrips
0x00002090 692e2e2e 005b2a5d 204d656e 64656b72 i....[*] Mendekr
0x000020a0 69707369 20646174 612e2e2e 00000000 ipsi data.....
0x000020b0 5b2d5d20 50617373 776f7264 2053616c [-] Password Sal
0x000020c0 61682e20 50726f67 72616d20 62657268 ah. Program berh
0x000020d0 656e7469 2e00 ent..
```

Seperti nya ini adalah output yang dikeluarkan program ketika dijalankan. Maka disini saya menggunakan command satu laginya `readelf -x .data enigma_box | less -S` dan ketika di jalankan mengeluarkan output berikut.

```
Hex dump of section '.data':
0x00004080 00000000 00000000 88400000 00000000 .....@.....
0x00004090 00000000 00000000 00000000 00000000 .....
0x000040a0 236a5722 b991a153 77f85b7a 91007913 #jW"...Sw.[z..y.
0x000040b0 9f0d10b7 c137cab1 6ef75eb0 3ecbb962 .....7..n.^.>..b
0x000040c0 ea0e45d1 90938b08 532b7905 b5878c81 ..E.....S+y.....
0x000040d0 aabbccdd eeff0011 22334455 66778899 ..... "3DUfw..
```

Dari isi nya keliatannya sudah terenkripsi.

4. Maka selanjutnya saya disini mencari offset file (file offset) untuk array yang ditemukan.

Jika tadi `readelf -x/objdump -s` memberi alamat seperti `00000040 aa bb cc ...`, itu adalah offset dalam section. Kita butuh offset file (bukan virtual address runtime). Cara mudah: gunakan `strings -t x` untuk melihat offset file ASCII strings — untuk byte arrays kita gunakan `readelf` lalu hitung file offset dengan `readelf -S + section offset`. Proses singkat:

- Lihat section headers untuk mengetahui Offset dari section `.data/.rodata`:

```
readelf -S enigma_box | egrep '\.data|\.rodata' -n
```

Ketika dijalankan mengeluarkan output berikut.



```
readelf -S enigma_box | egrep '\.data|\.rodata' -n
38: [16] .rodata PROGBITS 0000000000000200 00002000
56: [25] .data PROGBITS 0000000000000408 00003080
```

- Di `readelf -x .data` jika dilihat alamat relatif dalam section (baris awal biasanya menampilkan alamat `0x0` jika `readelf` menampilkan offset relatif ke section). Untuk file offset absolut = `section_offset + offset_in_readelf`.

Contoh alur (format umum):

- `readelf -S` menunjukkan `.data` offset = `0x640`.
 - `readelf -x .data` menampilkan byte dengan address `0x0: ...` atau `0x10: ...` yang berarti relative; jika Anda ingin file offset byte pertama tampil: `file_offset = 0x640 + 0x0 = 0x640`.
5. Lalu disini saya ekstrak byte array langsung ke file (menggunakan `dd`)
Setelah Anda menentukan `file_offset` (hex) dan panjang `len` (decimal), extract:

```
file_offset=0x640
```

```
len=48
```

```
dd if=enigma_box of=ciphertext.bin bs=1 skip=$((file_offset)) count=$len status=none
xxd -p ciphertext.bin
```

Lalu mengekstrak IV (mis. 16 bytes) dari offset lain:

```
iv_offset=0x630
```

```
dd if=enigma_box of=iv.bin bs=1 skip=$((iv_offset)) count=16 status=none
xxd -p iv.bin
```

Untuk apa? Untuk memastikan kita punya ciphertext & IV dalam format biner murni untuk didekripsi.

6. Lalu disini saya memverifikasi ciphertext length & struktur (harus kelipatan 16 untuk AES-CBC)

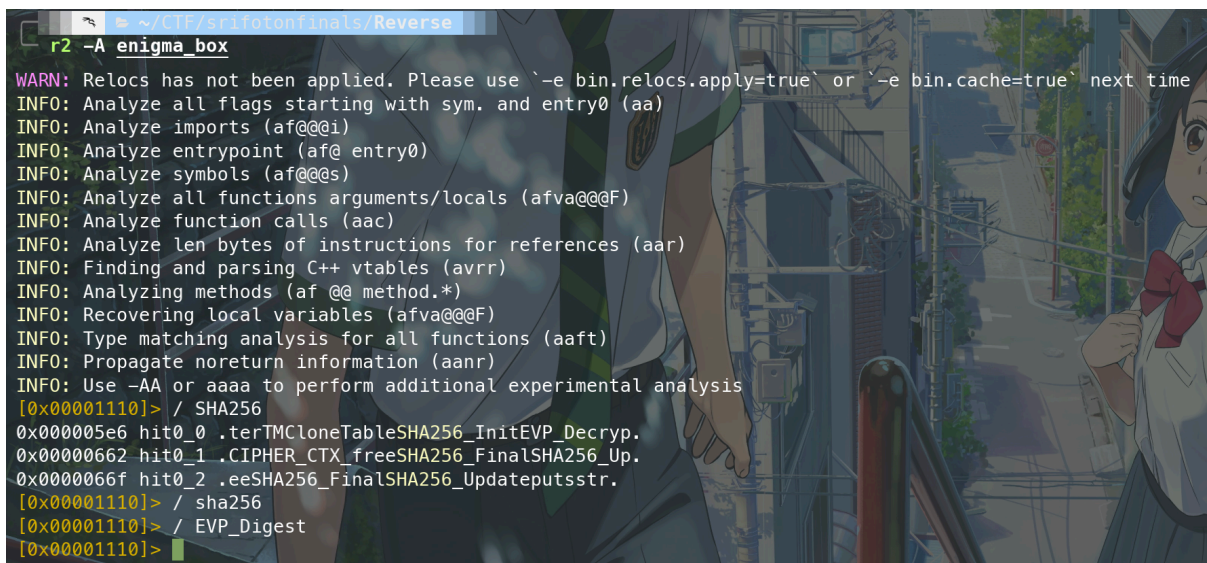
Dengan menggunakan command berikut :

```
wc -c ciphertext.bin
```

Jika bukan kelipatan 16, mungkin ciphertext sudah diencode (base64) — cek dengan file atau strings. Namun di challenge ini biasanya kelipatan 16.

7. Mencari logika derivasi key (reverse):

Disini saya menggunakan r2 singkat (agar cepat) Ketika saya jalankan mengeluarkan output berikut :



```
r2 -A enigma_box
WARN: Relocs has not been applied. Please use '-e bin.relocs.apply=true' or '-e bin.cache=true' next time
INFO: Analyze all flags starting with sym. and entry0 (aa)
INFO: Analyze imports (af@@@i)
INFO: Analyze entrypoint (af@ entry0)
INFO: Analyze symbols (af@@@s)
INFO: Analyze all functions arguments/locals (afva@@@F)
INFO: Analyze function calls (aac)
INFO: Analyze len bytes of instructions for references (aar)
INFO: Finding and parsing C++ vtables (avrr)
INFO: Analyzing methods (af @@ method.*)
INFO: Recovering local variables (afva@@@F)
INFO: Type matching analysis for all functions (aaft)
INFO: Propagate noreturn information (aanr)
INFO: Use -AA or aaaa to perform additional experimental analysis
[0x00001110]> / SHA256
0x000005e6 hit0_0 .terTMCloneTableSHA256_InitEVP_Decryp.
0x00000662 hit0_1 .CIPHER_CTX_freeSHA256_FinalSHA256_Up.
0x0000066f hit0_2 .eeSHA256_FinalSHA256_Updateputsstr.
[0x00001110]> / sha256
[0x00001110]> / EVP_Digest
[0x00001110]>
```

Disini saya menemukan Hal yang menarik yaitu SHA256 disitu terdapat tulisan decrypt dan FinalSHA256_Up.

8. Maka disini saya selanjut menghitung SHA256(password) dari hint / password kandidat karena punya password yang disebut (contoh: Pa55_w0rd_g3n1us!), maka hitung:

```
echo -n 'Pa55_w0rd_g3n1us!' | sha256sum
```

dan ketika mengeluarkan hasil nya maka hasilnya akan disimpan ke key_hex.

9. Maka langkah selanjutnya saya membuat script agar mendapatkan langsung mendapatkan flagnya dengan bentuk kode berikut.

```
PY huntflag.py > ...
1  from Crypto.Cipher import AES
2  from hashlib import sha256
3
4  password = b"Pa55_w0rd_g3n1us!"
5  key_hex = sha256(password).hexdigest()
6  iv_hex = "aabbccddeeff00112233445566778899"
7  ct_hex = "236a5722b991a15377f85b7a910079139f0d10b7c137cab16ef75eb03ecbb962ea0e45d190938b08532b7905b5878c81"
8
9  key = bytes.fromhex(key_hex)
10 iv = bytes.fromhex(iv_hex)
11 ct = bytes.fromhex(ct_hex)
12
13 cipher = AES.new(key, AES.MODE_CBC, iv)
14 pt = cipher.decrypt(ct)
15
16 pad = pt[-1]
17 if 1 <= pad <= 16 and pt.endswith(bytes([pad])*pad):
18     pt = pt[:-pad]
19 else:
20     print("Warning: padding looks wrong (proceeding without trim).")
21
22 print("key_hex:", key_hex)
23 print("plaintext:", pt.decode('utf-8', errors='replace'))
24
```

10. Dan ketika dijalankan kode tersebut mengeluarkan output berikut.

```
PS C:\Users\ariq\Documents\srifotonfinals> & C:/Users/ariq/AppData/Local/Micros
lag.py
key_hex: e2d180de1ff12c366e52cff150978deaa51f5ea69407eef4aa4f4d7aa877db07
plaintext: SRIFOTON{R3v3rs3_CrYpt0_4nd_L0g1c_1s_H4rd!}
```

11. Maka Flag dari chall tersebut adalah:

SRIFOTON{R3v3rs3_CrYpt0_4nd_L0g1c_1s_H4rd!}